CENTRO UNIVERSITÁRIO DO DISTRITO FEDERAL – UDF

COORDENAÇÃO DO CURSO DE SISTEMAS DE INFORMAÇÃO

Anna Beatriz Resende Oki Corrêa

Desafios da modernização de sistemas legados com agentes de IA no desenvolvimento de software

BRASÍLIA

2026

Anna Beatriz Resende Oki Corrêa

Desafios da modernização de sistemas legados com agentes de IA no desenvolvimento de software

Trabalho de conclusão de curso apresentado à Coordenação do Curso de Sistemas de Informação, do Centro Universitário do Distrito Federal - UDF, como requisito parcial para obtenção do grau de bacharel / tecnólogo em Sistemas de Informação.

Orientador: Prof. Dr. Xxxx

BRASÍLIA

?Sobrenome?, ?Nome?. Título : ?author?. – Brasília, 2026. 34 f.

Orientador: ?orientador?. Trabalho de Conclusão de Curso (Graduação – Sistemas de Informação) – Centro Universitário do Distrito Federal – UDF. Coordenação do Curso de Sistemas de Informação, Brasília, DF, 2026.

1. ?Assunto?. 2. ?Assunto?. 3. ?Assunto?. I. ?Título?.

CDU: ?consultar na biblioteca?

?Titulo?

Trabalho de conclusão de curso apresentado à ?Coordenação?, do Centro Universitário do Distrito Federal – UDF, como requisito parcial para obtenção do grau de bacharel / tecnólogo em ?Curso?.

Orientador: ?orientador?.

Brasília, _____ de _____ de 2026.

_____

NOME DO EXAMINADOR
Titulação
Instituição a qual é filiado

_____

NOME DO EXAMINADOR
Titulação
Instituição a qual é filiado

_____

NOME DO EXAMINADOR
Titulação
Instituição a qual é filiado

Nota: _____

Dedico este trabalho à minha família, pelo apoio, incentivo e compreensão durante toda a minha trajetória acadêmica.

Agradecimentos

Agradeço, primeiramente, a Deus, por mais uma conquista; ao meu orientador, pela dedicação e correções; e aos bibliotecários pelo suporte em todas as pesquisas.

"A educação é a arma mais poderosa que você pode usar
para mudar o mundo."

Nelson Mandela

Resumo

Este trabalho trata da atualização de sistemas legados para novas tecnologias com o uso de agentes de IA para desenvolvimento. O objetivo foi identificar os desafios desse uso, do ponto de vista do desenvolvimento. Para isso, foi feita uma revisão de estudos recentes sobre o tema, e os desafios encontrados foram organizados em categorias. Depois, foi feito um estudo de caso com um sistema real, feito em Power Apps. Com a ajuda de um agente de IA, quatro regras desse sistema foram convertidas para Python. Os resultados do código novo foram comparados, por testes, com os de um app de teste do Power Apps, que rodou as fórmulas originais com dados inventados. Nos casos testados, os resultados foram iguais. Mesmo assim, a leitura das fórmulas mostrou comportamentos escondidos nas regras, como uma regra que não ignora os registros excluídos e uma mesma conta feita de dois jeitos. Neste estudo, o desafio mais observado foi o das regras escondidas. Como foram poucas regras e os dados foram inventados, esses resultados valem só para o caso estudado.

Palavras-chave: *Sistemas legados. Modernização de software. Agentes de IA. Desenvolvimento de software.*

Abstract

This work addresses the update of legacy systems to new technologies using AI development agents. The objective was to identify the challenges of this use from a development perspective. To do so, a review of recent studies on the subject was carried out, and the challenges found were organized into categories. Then, a case study was carried out with a real system built in Power Apps. With the help of an AI agent, four rules of this system were converted to Python. The results of the new code were compared, through tests, with those of a Power Apps test app that ran the original formulas with made-up data. In the tested cases, the results were the same. Even so, reading the formulas revealed hidden behaviors in the rules, such as a rule that does not ignore deleted records and the same calculation done in two different ways. In this study, the most observed challenge was hidden rules. Since only a few rules were converted and the data was made up, these results apply only to the case studied.

Keywords: *Legacy systems. Software modernization. AI agents. Software development.*

Lista de ilustrações

Sumário

1 INTRODUÇÃO 12

2 OBJETIVOS 15
2.1 Objetivo geral 15
2.2 Objetivos específicos 15
2.3 Delimitação do trabalho 16
2.4 Contribuição esperada 16

3 FUNDAMENTAÇÃO TEÓRICA 17
3.1 Sistemas legados e modernização 17
3.2 Agentes de IA no desenvolvimento 18
3.3 Conversão de código e verificação 18
3.4 Desafios encontrados na literatura 19

4 METODOLOGIA 21
4.1 Tipo de pesquisa 21
4.2 O estudo de caso 21
4.3 Descrição do sistema atual 22
4.3.1 Finalidade e usuários 22
4.3.2 Tecnologia e estrutura 22
4.3.3 Fluxo de um pedido 22
4.3.4 Regras do sistema 22
4.3.4.1 Registros excluídos 23
4.3.4.2 Diárias nacionais por cargo 23
4.3.4.3 Valor de cada linha de custo 23
4.3.4.4 Total previsto de custos 23
4.3.4.5 Total realizado 24
4.3.5 Como as regras aparecem no código 24
4.4 Dados sintéticos 24
4.5 Como os resultados esperados foram obtidos 25
4.6 Uso do agente de IA 25
4.7 Como foi a avaliação 26

5 RESULTADOS E CONSIDERAÇÕES FINAIS 27
5.1 O que foi convertido 27
5.2 Resultado dos testes 28
5.3 Desafios observados 28
5.4 O que o desenvolvedor pode fazer 30
5.5 Limitações do estudo 31
5.5.1 Pontos não verificados 31

6 CONCLUSÃO 33

REFERÊNCIAS 34

1 Introdução

Muitas organizações ainda usam sistemas que foram criados há muitos anos e que continuam sendo parte do trabalho do dia a dia. Esses sistemas são chamados de sistemas legados. Eles não são assim chamados só pela idade, mas porque a organização depende deles, e porque mantê-los, evoluí-los ou integrá-los a soluções mais novas se tornou difícil, já que foram feitos com tecnologias e ferramentas que já não acompanham as práticas atuais. Ao longo dos anos, esses sistemas recebem alterações e passam a guardar regras de negócio importantes, isto é, as regras que definem como a organização trabalha, como o cálculo de um valor ou a aprovação de um pedido. Em um sistema bem cuidado, essas regras estão descritas na documentação, que é o material que explica como ele funciona, como manuais, descrições das regras e comentários no código. Mas os próprios estudos sobre sistemas antigos apontam que esse material nem sempre existe ou se mantém em dia. Alves, Politowski e Montandon (2026) listam, entre as dificuldades de manter sistemas legados, a documentação técnica ausente ou desatualizada, e Diggs et al. (2024) afirmam que o código legado frequentemente não tem documentação precisa e suficiente. A partir disso, é possível concluir que, em sistemas antigos, a documentação pode estar incompleta ou desatualizada, e que, nesses casos, a única fonte do que o sistema realmente faz é o próprio código. Quem precisa entender uma regra tem então de descobri-la lendo o código, o que é lento e sujeito a erros. Por isso, parte da pesquisa recente trata de como ajudar nessa tarefa, e Diggs et al. (2024) testaram o uso de modelos de linguagem, programas de IA que entendem e escrevem texto e código, para gerar a documentação de código antigo.

Diante dessas dificuldades, é natural perguntar por que as organizações não substituem simplesmente o sistema antigo por um novo. Os estudos dão algumas pistas. Alves, Politowski e Montandon (2026) afirmam que sistemas legados sustentam processos críticos do negócio ao longo do tempo e se tornaram grandes demais para serem substituídos ou reescritos do zero, e que reescrever à mão milhões de linhas de código pode introduzir diferenças de comportamento em relação ao sistema original. Diggs et al. (2024) acrescentam que muitos desses sistemas têm funções críticas, nas quais um erro ou uma mudança inesperada de funcionalidade pode trazer riscos importantes. Por isso, em vez de trocar tudo de uma vez, a alternativa costuma ser a modernização, que é a atualização do sistema para tecnologias mais novas, preservando o que ele já faz.

A modernização pode acontecer de duas formas. A primeira é a conversão do sistema para outra linguagem de programação, que é a forma de escrever as instruções que o computador executa. A segunda é a atualização de uma versão antiga de uma tecnologia para uma mais nova, mantendo a mesma linguagem. Um exemplo da segunda é a migração de projetos em Java, uma linguagem de programação muito usada, da versão 8 para a 17, que May et al. (2025) apontam como uma atualização trabalhosa, porque cada nova versão do Java traz incompatibilidades com as

anteriores. Em qualquer uma das duas formas, não basta reproduzir as telas e as funcionalidades em uma tecnologia recente. É preciso entender o que o sistema antigo faz, inclusive regras que não estão escritas em lugar nenhum, e garantir que o sistema novo continue fazendo o mesmo. Isso envolve ler o código, descobrir essas regras e validar o resultado.

Estudos recentes mostram como esse trabalho acontece na prática. Um deles é o de Alves, Politowski e Montandon (2026), que trabalharam com um sistema de gestão de shopping centers escrito em Visual Basic 6, uma linguagem antiga, hoje descontinuada, que já não recebe atualizações. O sistema estava em produção havia mais de vinte anos e é organizado em um núcleo e em módulos, que são partes separadas do sistema, cada uma responsável por um assunto. Os autores escolheram dois módulos de naturezas diferentes: um de medições, voltado a cálculos como consumo e tarifas, e um de compras, que depende de transações e de fluxos de aprovação. Esses módulos foram migrados, ou seja, reescritos em outra linguagem, o C#, mantendo o mesmo banco de dados, e quem fez a migração foi um agente de IA. A migração envolveu 12 funcionalidades, isto é, tarefas que o sistema executa, classificadas pelos autores como simples, médias e complexas, e o objetivo era que o código novo se comportasse como o original.

Outro exemplo vem da IBM. Chakaravarthy et al. (2026) tratam de aplicações escritas em COBOL, uma linguagem criada há várias décadas e ainda usada em sistemas de grandes empresas, que se tornam difíceis de manter e de levar para a nuvem. Para convertê-las para Java, uma linguagem mais usada hoje, os autores combinam duas técnicas. A primeira é a análise do código, que é a leitura automática do programa para descobrir como ele está organizado, quais dados ele usa e em que ordem as instruções são executadas. A segunda é um modelo de linguagem, um programa de IA treinado para entender e escrever texto e código. A análise indica como o código em Java deve ser estruturado, e o modelo faz a tradução da lógica. A ferramenta foi avaliada em programas de teste e em aplicações de clientes e, segundo os autores, já é usada em ambientes de clientes da IBM.

No primeiro exemplo, a tarefa foi feita por um agente de IA. Um agente de IA para desenvolvimento é uma ferramenta que, além de responder perguntas, trabalha dentro do projeto: ele lê os arquivos do código, escreve e altera arquivos, executa comandos e roda testes, repetindo esses passos até chegar a um resultado. Testes automáticos são pequenos programas que conferem se o código faz o que deveria, comparando o resultado obtido com o esperado. Por isso o agente consegue verificar o próprio trabalho, mas por isso também o desenvolvedor precisa conferir se os testes são bons. Estudos recentes avaliaram esses agentes em tarefas de modernização, como a migração de um sistema para outra linguagem ou a atualização de uma versão antiga de uma tecnologia (LI et al., 2026; LIU et al., 2026; MAY et al., 2025).

Esses estudos mostram que usar agentes de IA na modernização é possível, mas os resultados variam: o desempenho muda de um sistema para outro, e há casos em que o agente deixa passar regras do sistema antigo. Além disso, cada estudo trata de um sistema, de uma linguagem e

de um agente diferentes, e a maior parte usa conjuntos de testes públicos ou sistemas de empresas. Encontramos pouco sobre como é usar o agente no dia a dia de quem desenvolve, em um sistema real e pequeno, e sobre o que acontece quando o desenvolvedor precisa conferir o que o agente fez. Os detalhes desses limites ficam para o Capítulo 3. O que importa aqui é que eles deixam uma lacuna, e é ela que este trabalho observa.

Para isso, este trabalho faz duas coisas. Primeiro, reúne o que a literatura relata sobre as dificuldades desse uso. Depois, faz um estudo de caso: quatro regras de um sistema real, feito no Microsoft Power Apps, uma plataforma para criar aplicativos com poucas linhas de código, foram convertidas para a linguagem Python por um agente de IA. Os resultados do código novo foram comparados, por testes, com os do sistema original, rodados com dados inventados. Com isso, este trabalho busca responder à seguinte pergunta:

> Quais são os desafios para converter e atualizar sistemas legados para novas tecnologias por meio de agentes de IA, do ponto de vista do desenvolvimento?

Este trabalho está organizado em seis capítulos, contando esta introdução. O Capítulo 2 apresenta os objetivos. O Capítulo 3 reúne o que a literatura relata sobre sistemas legados, agentes de IA, verificação do código e os desafios desse uso. Com base nisso, o Capítulo 4 descreve como o estudo de caso foi feito, e o Capítulo 5 apresenta os resultados e as considerações finais. O Capítulo 6 traz a conclusão.

2 Objetivos

Este capítulo apresenta o que o trabalho pretende alcançar e o que fica fora dele. Como visto na introdução, os estudos sobre agentes de IA na modernização de sistemas legados relatam resultados que variam bastante, e há pouco registro do uso desses agentes no dia a dia do desenvolvedor. Os objetivos a seguir mostram como este trabalho busca responder à pergunta de pesquisa.

2.1 OBJETIVO GERAL

O objetivo geral deste trabalho é identificar e analisar os desafios de atualizar sistemas legados com a ajuda de agentes de IA, do ponto de vista do desenvolvimento. Ou seja, o foco está no trabalho de quem desenvolve: o que exige cuidado, o que pode dar errado e como conferir o resultado do agente.

Para isso, o trabalho parte de duas perguntas menores. A primeira é o que a literatura já relata sobre os desafios desse uso. A segunda é o que aparece quando esse uso é observado em um sistema real e pequeno.

2.2 OBJETIVOS ESPECÍFICOS

O primeiro objetivo específico é levantar na literatura os desafios já relatados. Para isso, o trabalho reúne estudos recentes que usaram agentes de IA ou modelos de linguagem em tarefas de modernização, como a conversão de linguagem e a atualização de versão. De cada estudo, são anotados o que foi testado, os resultados relatados e as dificuldades que apareceram. Esse levantamento está no Capítulo 3.

O segundo é organizar esses desafios em categorias. Os estudos usam sistemas, linguagens e agentes diferentes, e por isso as dificuldades aparecem de formas diferentes. Agrupar as parecidas, como tamanho do sistema, regras escondidas e atalhos do agente, facilita a comparação e serve de base para o estudo de caso. O resultado é o quadro de desafios, também no Capítulo 3.

O terceiro é observar esses desafios em um estudo de caso. O trabalho propõe atualizar partes de um sistema real com a ajuda de um agente de IA, usando dados inventados. O resultado de cada parte é avaliado por testes que comparam o sistema novo com o atual, e cada problema

encontrado é registrado e classificado nas categorias do objetivo anterior. O método está no Capítulo 4, e os resultados, no Capítulo 5.

O quarto é discutir o que o desenvolvedor pode fazer para lidar com cada desafio. Com base na literatura e no estudo de caso, o trabalho aponta cuidados práticos, como atualizar em partes pequenas, proteger os testes e conferir se o agente tomou atalhos. Essa discussão aparece nos Capítulos 5 e 6.

2.3 DELIMITAÇÃO DO TRABALHO

O trabalho tem limites que ajudam a entender o que ele oferece. O estudo de caso observa um sistema, um agente e uma troca de tecnologia. Por isso, os resultados mostram o que aconteceu naquele caso e não valem para todos os sistemas. O trabalho também não compara agentes entre si, não mede o custo financeiro em detalhe e não avalia a qualidade do código além dos testes e da revisão do desenvolvedor.

2.4 CONTRIBUIÇÃO ESPERADA

Espera-se que o trabalho ajude o desenvolvedor a decidir quando usar um agente de IA na modernização de um sistema legado e quais cuidados tomar. Espera-se também deixar um método simples, que possa ser repetido em outros sistemas, como indicado nos trabalhos futuros.

3 Fundamentação Teórica

Este capítulo apresenta a base teórica do trabalho: o que são sistemas legados e modernização, como agentes de IA são usados nessa tarefa, como o resultado é verificado e quais desafios a literatura relata.

Esta revisão reúne estudos publicados entre 2024 e 2026 sobre modernização de sistemas legados com agentes de IA. A busca foi feita no Google e no Google Acadêmico, e teve o apoio de uma ferramenta de IA. Foram escolhidos os estudos que tratam de migração, atualização ou documentação de código legado com o uso de agentes ou modelos de linguagem e que relatam resultados ou dificuldades. Esta não é uma revisão sistemática, por isso pode não incluir todos os estudos da área.

3.1 SISTEMAS LEGADOS E MODERNIZAÇÃO

Um sistema legado é um sistema antigo que ainda está em uso. Ele pode estar escrito em uma linguagem pouco usada hoje, ter uma estrutura difícil de mudar ou depender de bibliotecas sem suporte. Mesmo assim, ele pode ser importante, porque muitas vezes guarda regras de negócio que a organização usa todos os dias.

Um exemplo é o sistema estudado por Alves, Politowski e Montandon (2026). Era um sistema de gestão em Visual Basic 6, em produção havia mais de vinte anos. Outro exemplo são os programas em COBOL, que a IBM migra para Java (CHAKARAVARTHY et al., 2026).

A modernização é o trabalho de levar esse sistema para uma tecnologia mais nova, sem perder o que ele já faz. Ela pode ser de dois tipos:

- **conversão de linguagem**, por exemplo de Visual Basic 6 para C#;
- **atualização de versão**, por exemplo de Java 8 para Java 17 (LIU et al., 2026; MAY et al., 2025).

Uma dificuldade relatada é a falta de documentação. Em alguns casos, o código é a única fonte do que o sistema faz. Diggs et al. (2024) testaram modelos de linguagem para documentar código legado em MUMPS e em assembly. Entender o código antigo é uma etapa importante da modernização.

3.2 AGENTES DE IA NO DESENVOLVIMENTO

Entender e atualizar um sistema legado dá trabalho. Agentes de IA são uma forma de apoiar esse trabalho. Um agente de IA para desenvolvimento é uma ferramenta baseada em um modelo de linguagem que faz mais do que responder perguntas. Ele lê os arquivos do projeto, edita o código, roda comandos e testes, e repete esses passos até chegar a um resultado.

Na modernização, o agente pode receber o código antigo e a tecnologia nova como alvo, e propor o código novo. O desenvolvedor trabalha junto: define a tarefa, revisa o resultado e decide o que aceitar.

Os agentes já foram testados nesse tipo de tarefa. Alves, Politowski e Montandon (2026) usaram o Claude Code para migrar módulos de um sistema em Visual Basic 6 para C#. May et al. (2025) e Liu et al. (2026) avaliaram agentes na atualização de projetos Java. Li et al. (2026) avaliaram agentes na modernização de repositórios em várias linguagens.

Outra linha de trabalho junta a IA com análise do código. A IBM combina análise estática do código COBOL com um modelo de linguagem para gerar código Java (CHAKARAVARTHY et al., 2026). A análise dá ao modelo informações sobre a estrutura do programa, e, segundo os autores, o resultado ficou melhor do que o de modelos usados sozinhos.

3.3 CONVERSÃO DE CÓDIGO E VERIFICAÇÃO

Quando um agente ou um modelo de linguagem gera código novo, é preciso conferir o resultado. Converter código não é só trocar a sintaxe. O código novo precisa fazer a mesma coisa que o antigo. Isso é chamado de equivalência funcional.

Por isso, Pan et al. (2024) consideram uma tradução correta só quando o código compila, roda e passa nos testes. Compilar não basta, porque um código pode compilar e mesmo assim se comportar de forma diferente.

Os testes também têm limites. Se o sistema tem poucos testes, erros podem passar sem ser vistos. No estudo de May et al. (2025), os projetos escolhidos têm alta cobertura de testes, e a cobertura não pode cair mais de 5 pontos percentuais depois da migração. No estudo de Li et al. (2026), os testes ficam escondidos do agente, para que ele não os use como atalho.

Outra forma de avaliar é pedir para outro modelo de linguagem julgar o resultado. A IBM usou esse método para medir a qualidade do código Java gerado (CHAKARAVARTHY et al., 2026). Mas, no estudo de Diggs et al. (2024), as métricas automáticas tiveram pouca relação com a nota dada por pessoas, e por isso os autores indicam a avaliação humana. Neste trabalho, a

verificação principal foi feita com testes.

3.4 DESAFIOS ENCONTRADOS NA LITERATURA

Além da verificação, há outros pontos a observar. A literatura relata resultados bons no uso de agentes de IA na modernização, mas também desafios. Eles estão resumidos no Quadro 1 e explicados a seguir.

Quadro 1 – Desafios da modernização com agentes de IA

Desafio	**O que os estudos relatam**	**Fonte**
Tamanho do sistema	Passou em 91,3% dos testes com menos de 10 mil linhas e em 15,3% com mais de 50 mil	Li et al. (2026)
Complexidade das funcionalidades	Equivalência de 92% (baixa), 81% (média) e 47% (alta)	Alves, Politowski e Montandon (2026)
Regras escondidas	O agente perdeu regras implícitas e regras espalhadas em vários arquivos	Alves, Politowski e Montandon (2026)
Atalhos do agente	Excluiu testes que falhavam e executou o programa original em vez de traduzi-lo	May et al. (2025); Li et al. (2026)
Linguagens antigas	Bons resultados em MUMPS, notas ruins em assembly, e alucinações em ambos	Diggs et al. (2024)
Custo	Em média, 3,81 milhões de tokens e US$ 4,19 por migração	Alves, Politowski e Montandon (2026)

Fonte: elaborado pela autora, com base nos trabalhos citados.

Tamanho e complexidade

Nos estudos lidos, os resultados foram piores em sistemas maiores e mais complexos. Além dos números do Quadro 1, Liu et al. (2026) relataram 71,67% de sucesso na migração mínima de Java 8 para Java 17 e 53,33% na migração máxima. No estudo de May et al. (2025), o melhor modelo migrou 52,3% dos projetos.

Regras que o agente não vê

No estudo de Alves, Politowski e Montandon (2026), o agente deixou de migrar 100 de 331 itens analisados. A maior causa, com 42 casos, foram módulos vizinhos que ele não detectou. As regras escritas de forma explícita foram migradas melhor do que as implícitas, como as que dependem de parâmetros opcionais ou de configurações globais.

Atalhos e falsa impressão de sucesso

O agente pode parecer ter acertado sem ter acertado. No estudo de May et al. (2025), um agente excluiu testes que falhavam do processo de construção do projeto. No estudo de Li et al. (2026), em uma execução sem regras contra atalhos, um agente chegou a 99,5% de testes aprovados, mas em cinco projetos ele só chamava o programa original. Por isso, o desenvolvedor deve proteger os testes e conferir o que o agente de fato fez.

Linguagens antigas e pouca documentação

Diggs et al. (2024) observaram que os modelos tendem a inventar informações em código legado. Os comentários gerados para MUMPS foram bem avaliados, mas os de assembly tiveram notas baixas.

Custo

O custo depende do tamanho da tarefa. No estudo de Alves, Politowski e Montandon (2026), as funcionalidades de alta complexidade usaram cerca de 9 milhões de tokens, em média, e a maior parte do gasto vem do que o agente lê, e não do que escreve.

Supervisão do desenvolvedor

Nos estudos lidos, o resultado do agente foi verificado de alguma forma. Alves, Politowski e Montandon (2026) usaram um segundo revisor para conferir as regras migradas. Diggs et al. (2024) indicam avaliação humana, porque as métricas automáticas não bastaram. E Chakaravarthy et al. (2026) relatam que juntar o modelo com análise do código melhorou os resultados e reduziu a variação entre eles.

O que ainda falta

Nos estudos que lemos, a avaliação usa conjuntos de testes públicos ou sistemas de empresas, e encontramos pouco sobre o uso do agente no dia a dia do desenvolvedor, em um sistema real e menor. É isso que o estudo de caso deste trabalho propõe.

4 Metodologia

Este capítulo descreve como o estudo de caso foi feito.

4.1 TIPO DE PESQUISA

Este trabalho tem abordagem qualitativa e caráter exploratório. Ele tem duas partes: uma revisão da literatura, apresentada no Capítulo 3, e um estudo de caso, descrito neste capítulo. A pesquisa é qualitativa porque descreve e analisa os desafios encontrados, sem medir com estatística. É exploratória porque busca conhecer melhor um tema ainda pouco observado na prática do desenvolvedor.

4.2 O ESTUDO DE CASO

Para observar na prática os desafios relatados na literatura, foi feito um estudo de caso com um sistema real de uma instituição pública. O sistema estudado foi o SIS Eventos, usado para registrar e aprovar a participação de pessoas em eventos. Ele foi feito no Microsoft Power Apps, com dados em listas do SharePoint. Neste estudo, partes dele foram convertidas para Python, com testes automáticos.

O número total de fórmulas do sistema não foi contado. Como referência do tamanho, a busca no editor encontrou 39 ocorrências da lista de diárias nacionais e 88 da quantidade de pessoas, em várias telas.

Foram convertidas quatro regras do sistema. As três primeiras foram planejadas desde o início, com graus de complexidade diferentes: o tratamento dos registros excluídos (regra 01, simples), o valor de cada linha de custo (regra 02, média) e o total previsto de custos (regra 03, complexa). A quarta, o total realizado (regra 04), foi feita a mais. Essas regras estão descritas na seção seguinte.

O responsável pelo sistema é o *stakeholder* do trabalho.

4.3 DESCRIÇÃO DO SISTEMA ATUAL

4.3.1 Finalidade e usuários

O SIS Eventos é usado em uma instituição pública para registrar e aprovar a participação de pessoas em eventos. Quem tem interesse em ir a um evento pela instituição cadastra um pedido no sistema, com dados como o evento, a data, a passagem e os valores. O gabinete usa o sistema para registrar e acompanhar esses eventos, e a aprovação é feita pelo presidente. O sistema também tem cadastros de áreas requisitantes, de pontos focais por área e de permissões dos usuários.

4.3.2 Tecnologia e estrutura

O sistema foi feito no Microsoft Power Apps e guarda os dados em listas do SharePoint. Ele tem nove telas: um painel de eventos por ano e por status, a lista de solicitações, o formulário para criar uma solicitação, as pendências, a prestação de contas, a configuração e os três cadastros. Os dados ficam em dez listas, como eventos, países, cargos, diárias nacionais, diárias internacionais e previsão de custos dos eventos.

4.3.3 Fluxo de um pedido

O pedido começa com o cadastro do evento. A pessoa informa dados como identificação, organizador, datas, tipo de evento, âmbito (nacional ou internacional), local, área requisitante, número do processo e justificativa. Junto com o pedido, ela registra os custos que espera ter, como passagem, diárias, seguro viagem, inscrição e custos complementares. O sistema guarda quem criou o pedido e quem fez a última alteração, com data e hora.

Cada pedido tem um status: Em Aberto, Em Análise, Aprovado, Não Aprovado, Cancelado ou Aprovado para ajustes. Neste último, o pedido volta para a pessoa corrigir. Depois da aprovação, os valores são pagos. Quando o evento termina, a prestação de contas compara o que foi previsto com o que foi realmente gasto.

4.3.4 Regras do sistema

Esta seção descreve as regras que foram lidas no código do sistema, no editor do Power Apps. Elas são a base para os testes do estudo de caso.

4.3.4.1 Registros excluídos

As listas têm um campo chamado Excluido. Os eventos, os custos e as diárias marcados com um asterisco (*) nesse campo são ignorados nas regras 01, 03 e 04. A regra 02 não olha esse campo: ela calcula o valor da linha de custo mesmo quando a linha está excluída, e usa o valor da diária da lista mesmo quando a diária está excluída.

4.3.4.2 Diárias nacionais por cargo

Em um evento nacional, a pessoa escolhe o cargo ou função. O sistema então mostra as diárias da lista de diárias nacionais que têm a mesma classificação desse cargo. Cada opção aparece com o nome da cidade ou capital e o valor da diária. A classe do cargo é preenchida automaticamente a partir da lista de cargos. Se o âmbito do evento não for nacional, essa lista não mostra opções.

4.3.4.3 Valor de cada linha de custo

O valor de cada linha de custo depende do tipo de custo, como mostra o Quadro 2. O sistema testa os casos em ordem e usa o primeiro que se aplica. O resultado aparece em reais.

Quadro 2 – Cálculo do valor de cada linha de custo

Tipo de custo	**Como o valor é calculado**
Seguro viagem	valor do seguro × quantidade de pessoas
Diária internacional	valor já convertido para reais, guardado na linha
Passagem	quantidade de pessoas × passagem individual
Diária de outro cargo (valor digitado)	quantidade de diárias × valor digitado × quantidade de pessoas
Diária nacional (da lista)	quantidade de diárias × valor da diária
Inscrição	valor já convertido para reais, guardado na linha
Custo complementar	o próprio valor

Fonte: elaborado pela autora, com base no código do sistema.

4.3.4.4 Total previsto de custos

O sistema calcula o total previsto de custos dos eventos aprovados de um ano. Para cada evento aprovado, ele soma as linhas de custo previstas que não foram excluídas. Cada linha soma sete partes:

- diária de outro cargo: quantidade de diárias × valor digitado × quantidade de pessoas;
- diária nacional: quantidade de diárias × valor da lista × quantidade de pessoas;
- diária internacional: cotação do dólar × valor em dólar × quantidade de diárias × quantidade de pessoas;
- inscrição: o valor, multiplicado pela cotação do dólar quando a moeda é o dólar;
- custo complementar: o próprio valor;
- seguro viagem: valor × quantidade de pessoas;
- passagem: passagem individual × quantidade de pessoas.

Cada parte é arredondada para duas casas decimais, e o total aparece em reais. Essa conta soma todos os campos da linha, sem olhar o tipo de custo.

4.3.4.5 Total realizado

O sistema também mostra o total realizado dos eventos aprovados de um ano. Ele soma o campo Realizado das linhas de custo que não foram excluídas, dos eventos aprovados e não excluídos daquele ano. As linhas com o campo Realizado vazio ficam de fora da soma.

4.3.5 Como as regras aparecem no código

As contas ficam dentro de controles das telas, como galerias, botões e campos de texto, e não em um lugar só. A busca no editor encontrou 39 ocorrências da lista de diárias nacionais e 88 da quantidade de pessoas, em várias telas. A fórmula que monta a lista de diárias nacionais tem três casos com as mesmas contas e condições diferentes. O editor também avisa que há controles com versões mais recentes disponíveis, como 20 controles na tela de exercícios. Esses pontos foram observados no estudo de caso, descrito no Capítulo 5.

4.4 DADOS SINTÉTICOS

O estudo de caso usou dados sintéticos, ou seja, dados inventados para o estudo. Nenhum dado real foi usado. Os dados ficaram em três tabelas, com os mesmos nomes de campos das fórmulas: eventos, linhas de custo e diárias nacionais.

Os casos foram escolhidos para passar por cada regra da descrição do sistema atual. Eles têm eventos de 2025 e 2026, com todos os status, eventos, linhas de custo e diárias marcados como excluídos, cada tipo de custo, inscrição em dólar e em real, e um evento sem custos. Todos os valores são inteiros.

4.5 COMO OS RESULTADOS ESPERADOS FORAM OBTIDOS

No sistema atual, os dados ficam em listas do SharePoint, e as contas ficam dentro de controles das telas do Power Apps, e não em um lugar só.

Para saber o resultado de cada caso no sistema atual, as fórmulas das quatro regras foram copiadas do editor do Power Apps. Depois, elas foram rodadas em um app novo e vazio do Power Apps, criado só para o teste. Nesse app, os dados inventados foram carregados em coleções com os mesmos nomes das listas, e as fórmulas foram coladas nos controles sem mudança. O app real e as listas do SharePoint não foram usados. A autora anotou o que o app mostrou em uma tabela de resultados esperados.

No código em Python, os mesmos dados foram processados pelo código feito pelo agente. O resultado de cada caso foi comparado com a tabela de resultados esperados. Quando os resultados foram iguais, a regra foi considerada equivalente nesse caso.

4.6 USO DO AGENTE DE IA

A conversão foi feita com o agente Claude Code, com o modelo Claude Opus 5.5, dentro do ambiente de desenvolvimento. O processo teve estes passos:

1. as fórmulas, os dados inventados e a tabela de resultados esperados ficaram em arquivos no projeto;
2. as quatro regras foram pedidas ao agente de uma vez;
3. o agente escreveu o código e os testes juntos. Ele podia escrever os testes, mas não podia mudar a tabela de resultados esperados nem os dados, e os testes tinham de ler os valores esperados dessa tabela, sem valores escritos à mão;
4. o agente rodou os testes. Se algum falhasse, ele devia corrigir o código, e não os testes, e contar quantas tentativas foram necessárias;
5. no fim, o agente escreveu um registro com o que entendeu de cada regra, o que errou, o que ficou ambíguo, quantas tentativas fez e se a regra passou.

O agente também recebeu duas orientações: ser fiel ao sistema antigo, inclusive nos comportamentos estranhos, sem “consertar” as regras; e usar o tipo `Decimal` do Python para os valores em dinheiro.

4.7 COMO FOI A AVALIAÇÃO

Cada regra foi considerada correta se o código rodasse e passasse nos testes (PAN et al., 2024), ou seja, se desse os mesmos resultados do app de teste nos casos testados. Além disso:

- os valores esperados vieram só da tabela de resultados anotada pela autora;
- o agente não podia mudar essa tabela nem os dados;
- os testes cobriram os casos de cada regra que estão na tabela.

Os comportamentos observados foram relacionados aos desafios do Quadro 1 quando a ligação era clara.

Este estudo tem limites. Ele observou um sistema, um agente e uma troca de tecnologia, então os resultados mostram o que aconteceu nesse caso e não valem para todos os sistemas. Estudos parecidos, como o de Alves, Politowski e Montandon (2026), também tratam seus resultados dessa forma. O agente pode dar resultados diferentes a cada execução, e a mesma pessoa conduziu e avaliou o estudo. Além disso, o sistema atual foi feito em Power Apps, e não em código escrito em arquivos, como os sistemas dos estudos citados. Os outros limites estão no Capítulo 5.

5 Resultados e Considerações Finais

Este capítulo mostra o que aconteceu no estudo de caso. Ele descreve o que foi convertido, o resultado dos testes, os desafios que observamos, o que o desenvolvedor pode fazer e os limites do estudo.

5.1 O QUE FOI CONVERTIDO

Neste estudo, quatro regras do SIS Eventos foram convertidas do Power Fx, a linguagem de fórmulas do Power Apps, para Python. As três primeiras foram planejadas desde o início, e a quarta foi feita a mais:

- regra 01: contagem dos eventos por ano e por status, sem os registros excluídos;
- regra 02: valor de cada linha de custo;
- regra 03: total previsto de custos dos eventos aprovados de um ano;
- regra 04: total realizado dos eventos aprovados de um ano.

As fórmulas foram copiadas do editor do Power Apps, uma por arquivo. A conversão foi feita pelo agente Claude Code, com o modelo Claude Opus 5.5. O código ficou em um arquivo Python, e os testes foram escritos com a ferramenta pytest.

Os dados usados são inventados. São três tabelas, com os mesmos nomes de campos das fórmulas: eventos, linhas de custo e diárias nacionais. Os casos cobrem os anos de 2025 e 2026 e 23 linhas de custo.

Para saber o resultado certo de cada caso, as fórmulas originais foram rodadas em um app vazio do Power Apps, criado só para o teste, com esses mesmos dados inventados. O app real não foi usado. A autora anotou o que o app mostrou em uma tabela de resultados esperados. Por exemplo, em 2026 o total previsto foi R$ 19.450,00 e o total realizado foi R$ 20.350,00. Os testes comparam a saída do código em Python com essa tabela.

5.2 RESULTADO DOS TESTES

Ao todo, foram 42 testes, e os 42 passaram. Nenhum teste falhou. Cada regra passou na primeira execução, ou seja, com uma tentativa só. O Quadro 3 mostra o resultado por regra.

Quadro 3 – Resultado dos testes por regra

Regra	Testes	Passaram	Tentativas
01 – eventos por ano	15	15	1
02 – valor da linha de custo	23	23	1
03 – total previsto	2	2	1
04 – total realizado	2	2	1
Total	42	42	–

Fonte: elaborado pela autora, com base no resultado do pytest.

Na regra 01, há 14 testes de contagem e um teste da ordem dos anos. Na regra 02, há um teste por linha de custo, e cada um confere o valor e o texto em reais. Nas regras 03 e 04, há um teste para 2025 e outro para 2026.

Os testes leem os valores esperados direto da tabela de resultados. Eles não têm valores esperados escritos à mão.

5.3 DESAFIOS OBSERVADOS

Mesmo com todos os testes passando, a leitura das fórmulas mostrou comportamentos que não estavam claros na descrição do sistema. Nos casos testados, o código em Python repete esses comportamentos, porque a ideia era ser fiel ao sistema antigo, e não corrigir as regras.

A regra 02 não olha o campo Excluido

A regra 02 calcula o valor da linha mesmo quando a linha está marcada como excluída. Nos casos testados, o custo 9 está excluído e mesmo assim deu R$ 800,00. Já as regras 01, 03 e 04 deixam os registros excluídos de fora. Ou seja, a regra dos registros excluídos não está em um lugar só, e cada fórmula trata isso de um jeito. Isso se liga ao desafio das regras escondidas do Quadro 1, que inclui regras espalhadas em vários lugares.

A diária nacional não é multiplicada pelas pessoas

Na regra 02, a diária nacional da lista é calculada como quantidade de diárias × valor da diária, sem multiplicar pela quantidade de pessoas. Nos casos testados, o custo 4 tem 2 pessoas e deu R$ 800,00, que é 2 diárias × 400. Esse comportamento só aparece quando se lê a fórmula. Por isso, ligamos esse ponto ao desafio das regras escondidas do Quadro 1.

A diária excluída ainda é usada

A regra 02 busca o valor da diária na lista de diárias sem tirar as que estão excluídas. Nos casos testados, o custo 5 usa uma diária excluída e deu R$ 1.000,00. A regra 03, por outro lado, ignora as diárias excluídas. É o mesmo caso da regra dos excluídos espalhada em várias fórmulas, e por isso também ligamos esse ponto às regras escondidas do Quadro 1.

O “-” da regra 04 nunca aparece

A regra 04 usa a função `Coalesce` para mostrar “-” quando não há valor. Mas o primeiro argumento sempre começa com “R$”, então nunca fica vazio, e o “-” não aparece. O código em Python também não devolve “-”. Não encontramos ligação direta desse ponto com os desafios do Quadro 1.

De onde vêm os valores convertidos

Na regra 02, a diária internacional e a inscrição não fazem conta. Elas só mostram dois campos já guardados na linha: `ValorConvertido` e `InscricaoConvertida`. Neste estudo, não sabemos de onde vêm esses valores. Eles podem ser calculados em outra parte do sistema, que não foi vista. Isso parece com o desafio das regras escondidas do Quadro 1, no qual o agente não percebeu partes vizinhas do sistema. Mas, como não sabemos a origem dos campos, essa ligação não pôde ser confirmada.

A mesma conta feita de dois jeitos

O valor das linhas de custo aparece em duas fórmulas: na regra 02, linha por linha, e na regra 03, dentro do total. As duas fazem a conta de jeitos diferentes. A regra 03 multiplica a diária nacional pelas pessoas e ignora a diária excluída, e a regra 02 não faz nenhuma das duas coisas.

Por isso, em 2026, a parte do evento 1 no total da regra 03 é 6.350, enquanto a soma das linhas desse evento na regra 02, sem o custo 9, é 6.550. Isso se liga ao desafio das regras escondidas do Quadro 1, porque a mesma regra está em mais de um lugar.

O texto em reais

O app mostra os valores como texto, no formato "R$ 1.234,56", que vem da função `Text()` do Power Fx. O código em Python precisou montar esse mesmo texto. Também ficou uma dúvida: quando a regra 02 não tem resultado, o app parece mostrar só "R$", porque o `Text()` de um valor vazio dá texto vazio. Esse caso não foi testado. Não encontramos ligação direta desse ponto com os desafios do Quadro 1.

5.4 O QUE O DESENVOLVEDOR PODE FAZER

A partir do que observamos neste estudo, sugerimos alguns cuidados. Eles vêm de um caso só e de poucas regras, então são sugestões, e não regras para qualquer sistema.

- **Listar onde cada regra aparece.** Neste estudo, a regra dos registros excluídos estava em várias fórmulas, e cada uma tratava isso de um jeito. Antes de converter, vale procurar todas as fórmulas que usam o campo `Excluido` e anotar quais olham esse campo e quais não olham.
- **Pedir ao agente que aponte regras duplicadas.** O valor das linhas de custo aparecia nas regras 02 e 03, com contas diferentes. Pedir ao agente que compare as fórmulas e mostre onde a mesma conta é feita de mais de um jeito pode ajudar a achar essas diferenças antes dos testes.
- **Perguntar à equipe a origem dos campos guardados.** Neste estudo, não soubemos de onde vêm `ValorConvertido` e `InscricaoConvertida`. Perguntar ao responsável pelo sistema antes de converter evita copiar um campo sem saber como ele é calculado.
- **Escrever testes com centavos e casos de borda.** Nos casos testados, todos os valores eram inteiros, e vários casos ficaram sem teste. Incluir valores com centavos, anos sem eventos aprovados e linhas sem nenhum caso que se aplique ajudaria a testar o arredondamento e os comportamentos que não apareceram.
- **Conferir o texto na tela.** O texto em reais foi montado a partir do formato das fórmulas, e não copiado da tela. Copiar alguns textos direto do app ajudaria a confirmar o formato, o caso do "R$" sozinho e o "-" da regra 04.

5.5 LIMITAÇÕES DO ESTUDO

Este estudo tem limites que precisam ser levados em conta.

Primeiro, os dados inventados e o roteiro do app de teste foram criados por nós, com a ajuda do mesmo agente que depois fez a conversão. Esse agente também fez antes as contas à mão dos totais. Ou seja, o agente teve acesso aos resultados esperados.

Segundo, a coluna com o texto mostrado na tela, como “R$ 19.450,00”, não foi copiada da tela. Ela foi montada a partir do formato das fórmulas, com os valores anotados pela autora.

Terceiro, todos os valores dos dados são inteiros. Por isso, nenhum caso testa centavos nem o arredondamento. Também ficaram sem teste:

- um ano que só tem eventos excluídos;
- um ano sem eventos aprovados;
- uma linha de custo com campos de mais de um tipo preenchidos;
- uma linha da regra 02 sem nenhum caso que se aplique, ou com o valor convertido vazio.

Quarto, foram convertidas só quatro regras de um sistema maior.

Quinto, o tempo gasto em cada etapa não foi registrado. Por isso, o custo em tempo do uso do agente não foi avaliado neste estudo.

Por tudo isso, o fato de todas as regras terem passado na primeira tentativa não permite concluir que a conversão é simples. Nos casos testados, o código em Python deu os mesmos resultados do app de teste. Mas os comportamentos descritos na Seção 5.3 só foram vistos porque as fórmulas foram lidas com atenção, e não porque algum teste falhou.

5.5.1 Pontos não verificados

Alguns pontos das regras ficaram sem confirmação neste estudo:

- não verificamos com o responsável pelo sistema se a diária nacional deve ser calculada por pessoa. Na regra 02, a conta da linha não multiplica pela quantidade de pessoas, mas na regra 03 o total multiplica;

- não verificamos de onde vêm os campos de valor convertido, `ValorConvertido` e `InscricaoConvertida`;
- não verificamos a origem da variável do ano (`VarAnoExercicioSelecionado`), usada nas regras 03 e 04. No app de teste, ela foi definida à mão.

6 Conclusão

Este trabalho teve como objetivo entender os desafios de atualizar sistemas legados com agentes de IA, do ponto de vista do desenvolvimento.

Na literatura, encontramos desafios em seis pontos: tamanho do sistema, complexidade das funcionalidades, regras escondidas, atalhos do agente, linguagens antigas e custo. Nos estudos lidos, o resultado do agente foi verificado de alguma forma, e os testes foram uma das formas usadas para isso.

No estudo de caso, quatro regras do SIS Eventos foram convertidas do Power Fx para Python, com o agente Claude Code. Os resultados esperados vieram de um app vazio do Power Apps, com dados inventados. Foram 42 testes, e os 42 passaram. Cada regra passou na primeira tentativa.

Mesmo assim, a leitura das fórmulas mostrou comportamentos escondidos. A regra 02 não olha o campo de registro excluído, não multiplica a diária nacional pelas pessoas e usa uma diária excluída. A mesma conta aparece nas regras 02 e 03, feita de jeitos diferentes. E não sabemos de onde vêm dois campos de valor convertido. Esses pontos não apareceram porque um teste falhou. Eles apareceram porque as fórmulas foram lidas com atenção.

Respondendo à pergunta de pesquisa: neste estudo, o principal desafio observado foi o das regras escondidas do Quadro 1. Nos casos testados, o agente deu os mesmos resultados do sistema atual. A dificuldade estava em saber se os testes cobriam o suficiente e em entender regras que estavam espalhadas ou que dependiam de partes do sistema que não foram vistas. Os outros desafios do quadro não foram observados ou não foram medidos neste estudo.

O estudo tem limites. Os dados foram inventados, e o agente que fez a conversão também ajudou a criar os dados e viu os resultados esperados. Foram só quatro regras de um sistema maior, e nenhum caso testou centavos ou arredondamento. Por isso, passar de primeira não permite dizer que a conversão é fácil ou difícil em geral. Alguns pontos das regras também ficaram sem confirmação, e eles estão listados na Seção 5.5.1.

Como trabalhos futuros, sugerimos converter mais regras do mesmo sistema, usar dados reais com a autorização do responsável, repetir o estudo com outros agentes e medir o tempo gasto em cada etapa.

Referências

ALVES, I. d. S. R.; POLITOWSKI, C.; MONTANDON, J. E. Legacy System Modernization with Coding Agents: A Case Study. 2026. ArXiv:2608.28972. Disponível em: <https://arxiv.org/abs/2608.28972>.

CHAKARAVARTHY, V. et al. Enterprise-scale COBOL-to-Java translation: LLMs augmented with program analysis. In: Proceedings of the 48th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP '26). Rio de Janeiro, Brasil: [s.n.], 2026.

DIGGS, C. et al. Leveraging LLMs for Legacy Code Modernization: Challenges and Opportunities for LLM-Generated Documentation. 2024. ArXiv:2411.14971. Disponível em: <https://arxiv.org/abs/2411.14971>.

LI, X. et al. RepoMod-Bench: A Benchmark for Code Repository Modernization via Implementation-Agnostic Testing. 2026. ArXiv:2602.22518. Disponível em: <https://arxiv.org/abs/2602.22518>.

LIU, L. et al. MigrationBench: Repository-level code migration benchmark from java 8. In: Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '26). Jeju, Coreia do Sul: [s.n.], 2026. ArXiv:2505.09569.

MAY, V. et al. FreshBrew: A Benchmark for Evaluating AI Agents on Java Code Migration. 2025. ArXiv:2510.04852. Disponível em: <https://arxiv.org/abs/2510.04852>.

PAN, R. et al. Lost in translation: A study of bugs introduced by large language models while translating code. In: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE '24). Lisboa, Portugal: [s.n.], 2024.